

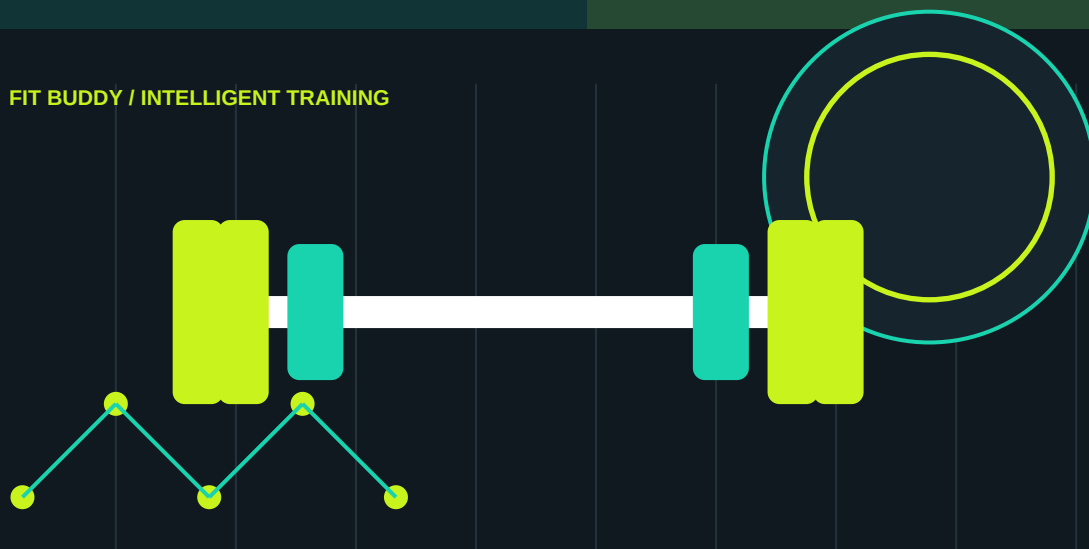
FIT BUDDY

2.0

AI FITNESS PLAN GENERATOR

A NEXT-GENERATION PROJECT SHOWCASE

FIT BUDDY / INTELLIGENT TRAINING



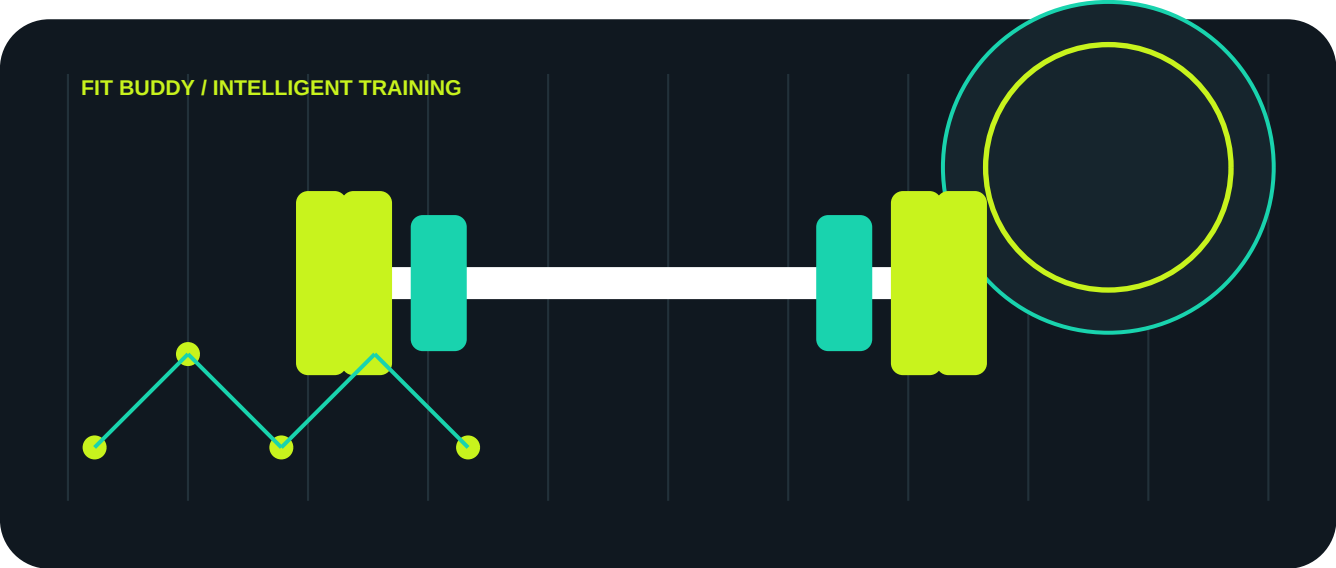
Flask • Gemini 2.5 Flash • SQLite • Chart.js • html2pdf.js

Original editorial + futuristic fitness visual language

01 / THE PRODUCT IDEA

From a fitness request to a usable training journey

Fit Buddy 2.0 is an AI-assisted fitness platform that turns a user's training target into a structured routine, supports conversational coaching, records activity and exposes progress views. The original report separates AI-generated content from the application layer that stores, formats and presents that content.



THE INPUT	A goal, target or fitness request gives the system a starting point.
THE INTELLIGENCE	Gemini 2.5 Flash is used for workout and conversation generation.
THE EXPERIENCE	Flask connects the AI/service layer with browser screens and stored activity.
THE MEMORY	SQLite provides the reported foundation for profiles and workout logs.

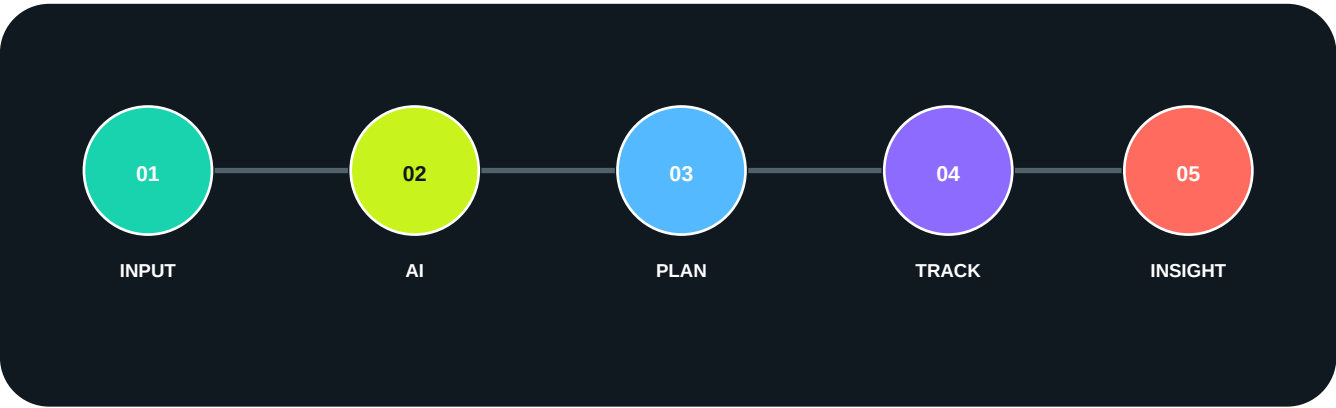
02 / WHY THE SYSTEM EXISTS

A fragmented workout experience becomes one connected flow

BEFORE	FIT BUDDY FLOW
Goal is expressed in natural language	Goal becomes a structured workout request
Workout content is generated as needed	AI output is shaped into expected fields
Activity can be difficult to organise	Workout logs create a history layer
Progress is easier to understand when visualised	Charts and summaries create a dashboard view
Routine can be shared/exported	Browser content can be converted into PDF

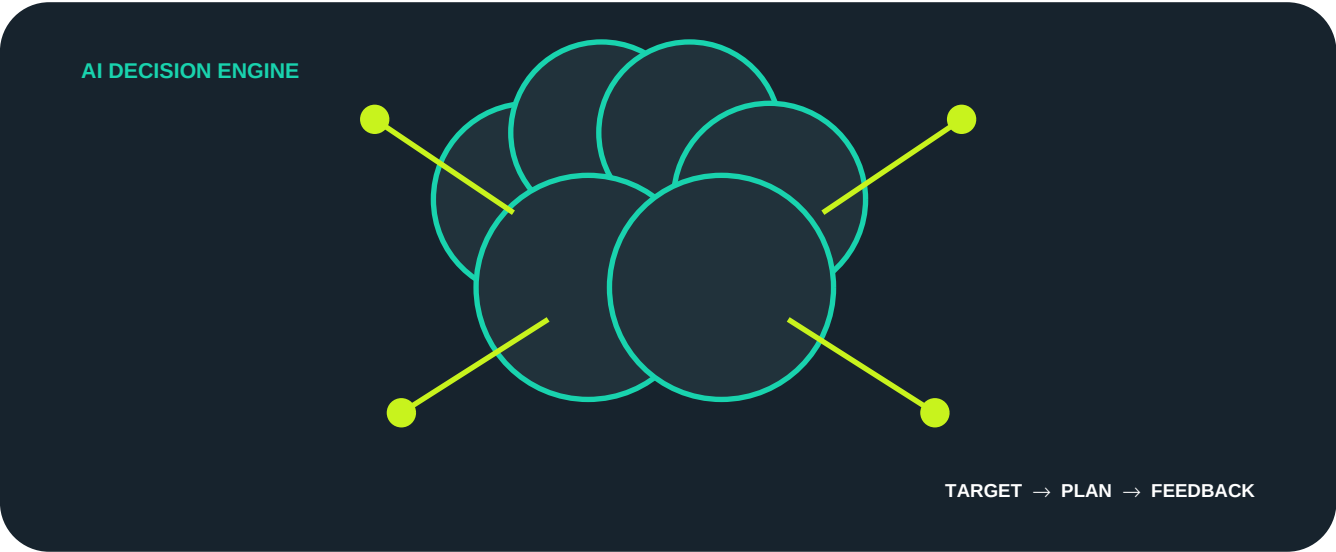
MODERN EXPLANATION

The project is easiest to understand as a closed loop: **intent** → **generation** → **presentation** → **activity log** → **progress view**. This framing explains the role of each technology without changing the documented architecture.



03 / THE AI LAYER

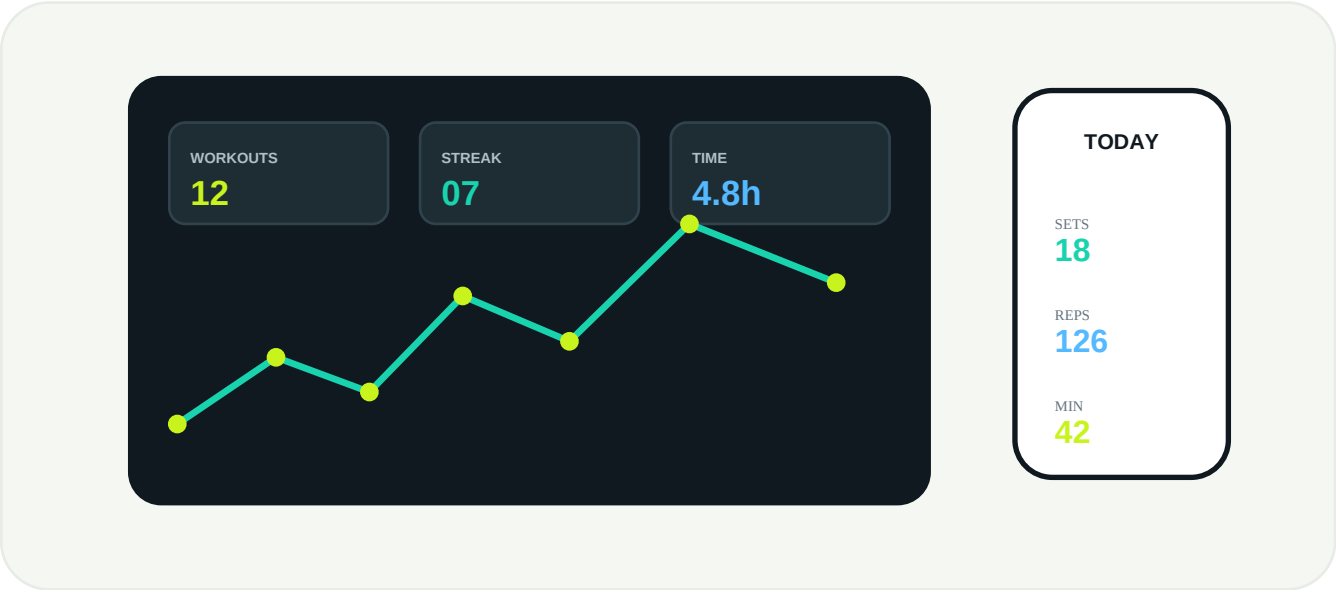
The AI engine: structured creativity, not uncontrolled text



The supplied implementation pattern sends a target to Gemini 2.5 Flash and requests JSON using a response schema. This matters because the browser needs predictable fields for rendering workout cards. The report also notes that model output should still be treated as untrusted input and validated before success is returned.

STAGE	WHAT HAPPENS	WHY IT MATTERS
1. Intent	Receive a target; the documented default is “Full Body” when no target is supplied.	Prevents an empty request from becoming an undefined workflow.
2. Prompt	The backend asks the model to create a workout for that target. Converts user intent into generated content.	
3. Schema	The generation request specifies JSON output and an expected response structure.	Makes downstream rendering more predictable.
4. Validation	Parse and validate the returned model content before treating it as successful.	Protects the application from malformed or incomplete output.
5. Presentation	Return JSON to the client so the interface can render the routine.	Separates AI generation from visual presentation.

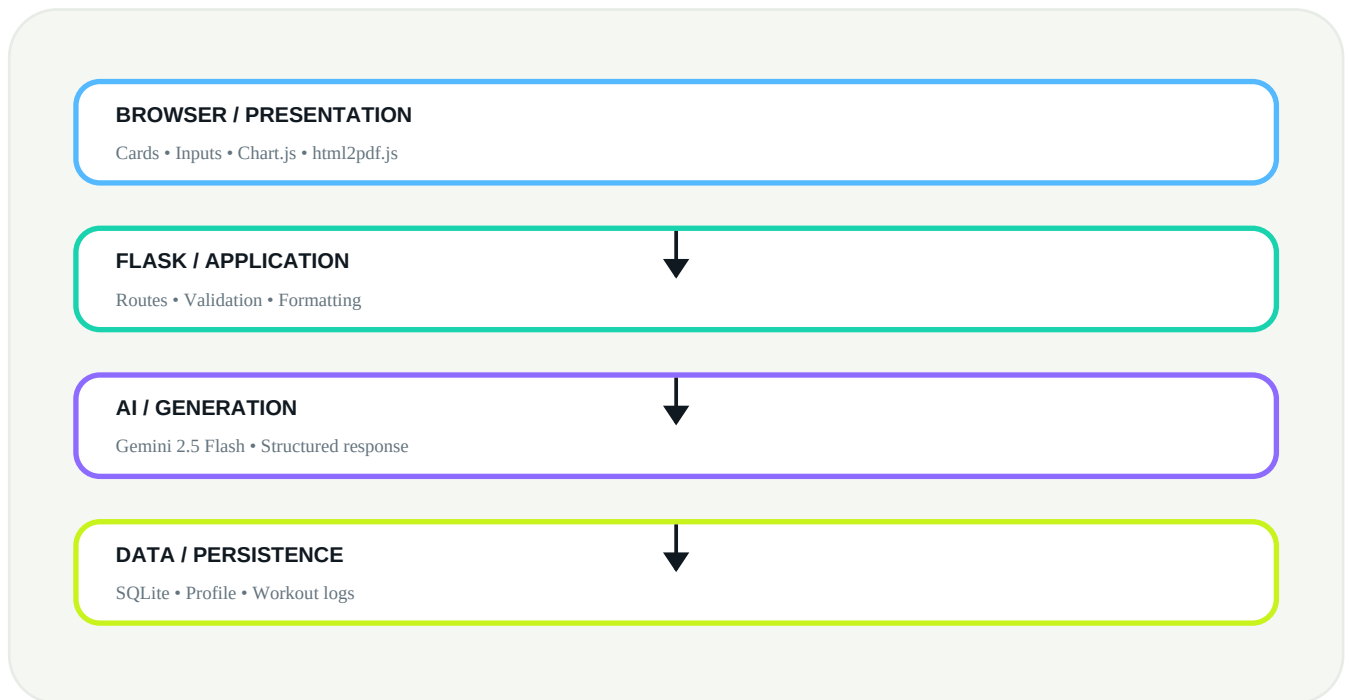
Designed around moments, not menus



	Discover User identifies the training target or asks a fitness question.
	Generate The backend requests structured AI content.
	Act The generated routine becomes the actionable workout view.
	Record A completed session can be stored in workout history.
	Reflect Charts, streaks and summaries make the history readable.
	Export The browser can turn a rendered routine into a downloadable PDF.

05 / SYSTEM BLUEPRINT

Four layers, one continuous product



The reported typical path is: browser request → Flask route → AI service or database → JSON response → interface update. The visual above turns that description into a presentation-ready blueprint.

06 / DATA LIFECYCLE

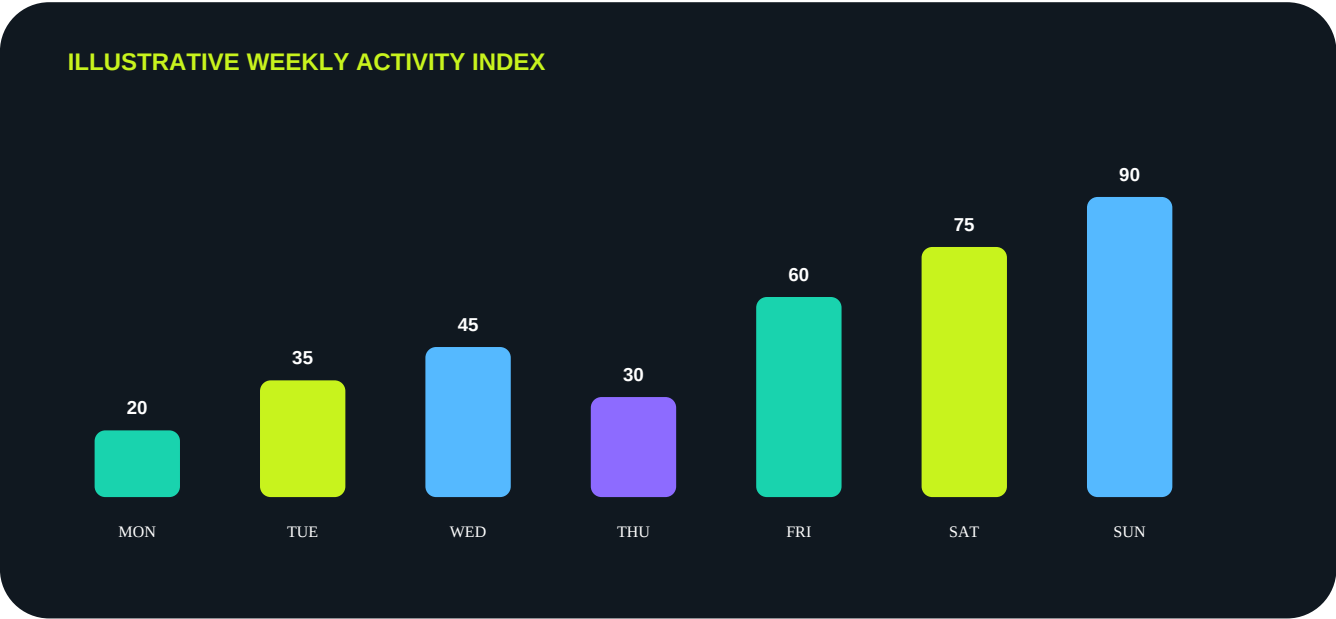
A small database with a clear growth path

ENTITY	REPORTED FIELDS	ROLE
user_profile	id • name • streak_count	Profile identity and consecutive-day count.
workout_logs	id • title • duration_min • calories_burned	Routine history and session metadata.

RECOMMENDED EVOLUTION

LINK	Connect workout logs to a user through a user_id foreign key.
TIME	Store a consistent completion timestamp, commonly UTC.
TRANSACTION	Write the log and update streak consistently.
ANALYTICS	Aggregate logs over defined time windows.

Turn stored sessions into understandable signals



The numeric chart is deliberately marked illustrative because the source report states that no underlying numeric dataset or formal metric definitions were supplied.

FREQUENCY	Completed sessions per day or week; separate planned from completed.
VOLUME	Sets × reps × load when load is actually captured; the listed schema does not contain load.
DURATION	Session minutes using consistent units.
STREAK	Consecutive calendar days with qualifying sessions, with timezone rules defined.
CALORIES	Recorded estimates; avoid presenting them as precise measurements.

08 / RELIABILITY & SECURITY

What makes an AI feature feel like a real application?

VALIDATE INPUT

Check JSON type, target length and allowed values.

VALIDATE AI OUTPUT

Parse and verify expected fields before rendering or persistence.

HANDLE FAILURES

Plan for timeouts, provider errors, quota limits and malformed responses.

PROTECT SECRETS

Keep API keys server-side; avoid exposing secrets or stack traces.

CONTROL OPERATIONS

Add request limits, safe logging and health monitoring.

TEST REAL FLOWS

Exercise missing inputs, repeated logs, timezone boundaries and export edge cases.

09 / TESTING

Reported results + a stronger verification plan

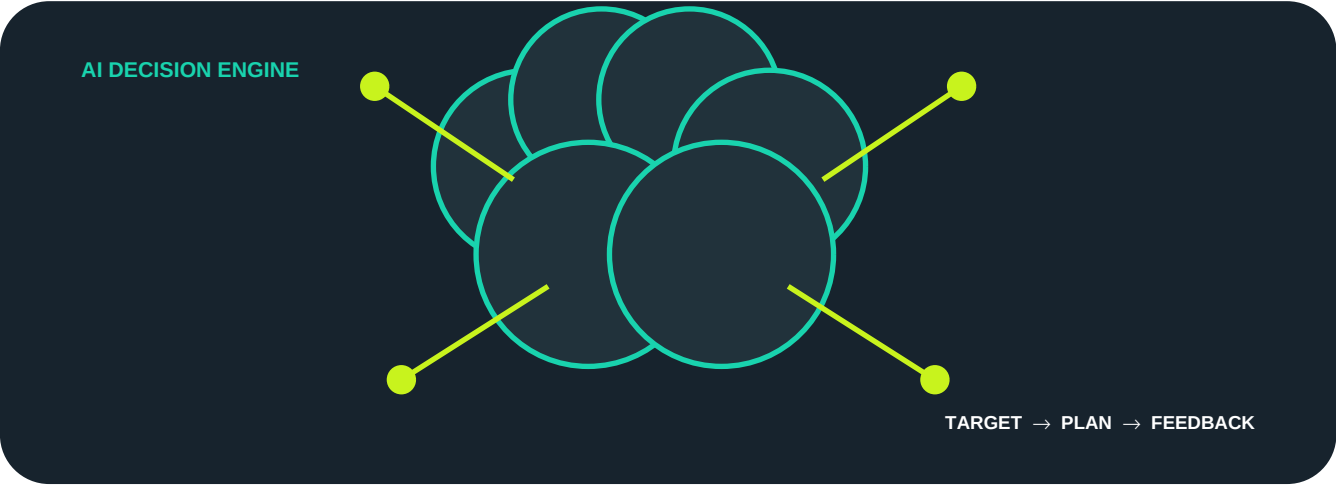
ID	AREA	ACTION	EXPECTED	RESULT
TC-01	Workout AI	POST /api/workout	Structured routine for UI cards	PASS*
TC-02	AI Coach	POST /api/chat	Natural-language coaching reply	PASS*
TC-03	Streak DB	POST /api/log-workout	Persist log and increment streak	PASS*
TC-04	PDF export	Client-side generation	Clean html2pdf.js document	PASS*

* PASS results are preserved as reported by the supplied project report; they were not independently executed during this redesign.

NEXT TEST LAYER

Schema edge cases • external-service failures • repeated logs • streak boundaries • timezone transitions • long PDF exports • special characters • unauthorised access • sanitisation • secret exposure • rate limiting.

Fit Buddy 2.0 at a glance

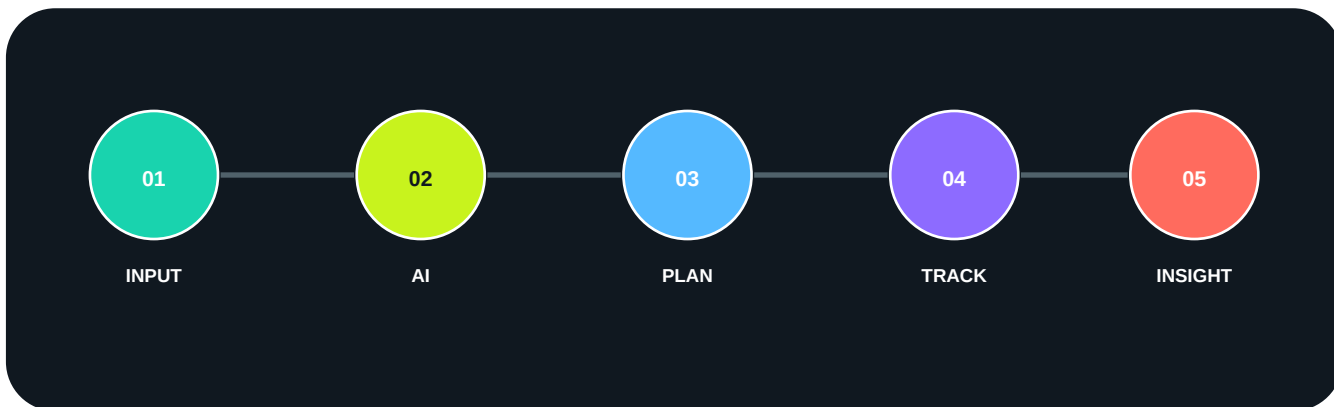


AI GENERATION	Goal-oriented routines and conversational support.
FLASK SERVICE	Application boundary for routes, formatting and validation.
SQLITE MEMORY	Profile and workout-log persistence.
VISUAL ANALYTICS	Charts and summaries for activity history.
PDF EXPORT	Browser-rendered routines converted into downloadable documents.

11 / CONCLUSION

Build the intelligence. Prove the workflow. Visualise the journey.

Fit Buddy 2.0 brings together structured AI generation, a Flask service boundary, SQLite-backed activity records, browser visualisation and PDF export. The most useful next step is not simply adding more features; it is making each existing stage measurable, validated and clearly documented.



01

Define analytics

Document formulas, units, periods and missing-data rules.

02

Enrich history

Capture fields required for meaningful volume and progression analysis.

03

Strengthen validation

Validate client input and model responses before persistence/rendering.

04

Harden failures

Add timeout, quota, malformed-response and database failure paths.

05

Document deployment

Record authentication, secret handling, rate limits, topology and monitoring.

The graphics in this edition are original vector illustrations created for this report. All charts labelled illustrative are presentation examples, not measured project data.